# Effectiveness of a Generate–Verify–Repair Pipeline with Batfish for LLM-Generated Vendor CLI Configurations

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We report a preregistered paired experiment (cnsms2026-001-gvr-batfish-prereg-v1) that evaluated whether a generate–verify–repair (GVR) pipeline—shared initial LLM candidate (gpt-5-mini) + a deterministic Batfish-style validator + at most one automated LLM repair—reduces deterministic-verifier detected semantic/safety failures on intent→vendor-CLI tasks. Planned N=300 pairs; 266 complete pairs were analysed after preregistered exclusions. Baseline pass-rate = 260/266 (97.744%); guarded (final GVR) pass-rate = 265/266 (99.624%). Paired difference = 0.01879699248; exact McNemar two-sided p = 0.0625. Paired bootstrap (B=10,000, seed=1729) 95% percentile CI = [0.0037593984962406013, 0.03759398496240601]. Repair was invoked for 6 tasks; median repair iterations per task = 0. Per-episode latency was not recorded in per-task artifacts and thus could not be evaluated. Under shared_initial_candidate semantics the observed absolute improvement is small and did not meet the preregistered $\alpha$=0.05 decision rule. We provide preregistered design details, deterministic validator semantics, exact paired counts, representative artifact-grounded repair examples, contamination findings (no exact public duplicates flagged), and a focused discussion of limitations and implications.**

## I. Introduction

Generative large language models (LLMs) are actively explored for NetOps tasks such as intent→CLI translation, zero-touch configuration synthesis, and automated maintenance workflows [1], and surveys emphasize both opportunity and risk when LLMs are applied to operational network configuration [2]. Stochastic LLM outputs can produce syntactic or semantic violations—missing required commands, unsafe command orderings, or constraint breaches—that create operational risk if applied directly. To mitigate that risk, pipelines that interpose deterministic verifiers (e.g., Batfish or header-space analyses) and bounded repair loops—collectively generate–verify–repair (GVR)—have been proposed to provide deterministic acceptance criteria and automated correction [3,4]. Prior work presents components and prototypes, but preregistered, paired evaluations that archive verifier traces and quantify verifier-triggered repair benefit on reproducible NetOps task banks remain limited.

This study tested the preregistered question (cnsms2026-001-gvr-batfish-prereg-v1): Does a GVR pipeline that uses a single sampled initial LLM candidate per task (shared_initial_candidate semantics), a deterministic Batfish-style validator, and at most one automated LLM repair call reduce deterministic-verifier detected semantic/safety failures relative to the same initial candidate without repair? Our principal contribution is a preregistered, artifact-archived paired evaluation executed under the CNSM Agentic AI Researcher constraints that reports exact paired counts, inferential outcomes, execution accounting, representative artifact examples, and an evidence-grounded interpretation.

## II. Related Work

Prior work relevant to this study spans (a) applied LLM prototypes and benchmarks for network configuration, (b) surveys documenting LLM failure modes and recommendations to ground generation, (c) proposals for generate–verify–repair and related architectures for deterministic validation, and (d) established deterministic network verification tools that serve as practical oracles in automated pipelines. We synthesize those threads and emphasize aspects most relevant to a preregistered paired GVR evaluation.

LLM-driven NetOps prototypes and benchmarks: Recent system-level and benchmark efforts investigate whether LLMs can aid network configuration generation and automation. NetConfEval and similar task collections examine intent→CLI translation and measure correctness under vendor-dialect constraints; these works provide public task exemplars and motivate reproducible task generators for controlled evaluation [1]. Such benchmark tasks typically encode required command sequences and vendor syntax, which directly map to the validator check types we archive in this study (required-sequence, allowed-field checks, topology reachability). Importantly, prior benchmarks often evaluate single-pass generation accuracy rather than integrated verifier-triggered repair, which motivates the paired GVR design adopted here.

Failure modes, grounding, and mitigation recommendations: Survey and synthesis papers summarize that LLMs commonly produce both syntactic and semantic errors when applied to domain-constrained tasks: missing commands, incorrect ordering, and hallucinated or unsafe modifications [2]. Those surveys recommend grounding strategies (retrieval, tool use), constrained generation, and verifier insertion to reduce the operational risk of accepting model outputs without deterministic checks. Our work adopts those prescriptions by interposing a deterministic validator and bounding the repair loop to one automated correction attempt, aligning experimentally with the practical mitigations advocated in the literature.

Generate–verify–repair and bounded repair proposals: The idea of interposing deterministic oracles and using closed-loop

repair is well established in proposals for regulated or safety-sensitive domains. Convergent GVR architectures aim to combine stochastic generation with deterministic acceptance criteria and limited repair to achieve practical correctness guarantees while preserving the generative model's productivity [3]. These proposals emphasize publishing verifier traces and repair prompts so that corrected outputs are auditable—an archival design we implemented in the execution artifacts (validator traces, repair_prompt_sha256, repair provider traces).

Deterministic network verifiers as oracles and their coverage limits: Tools like Batfish and header-space analyses provide deterministic, rule-based checks for reachability, ordering, and configuration invariants and have been incorporated into research prototypes that integrate model outputs with formal validation [4]. Batfish-style validators excel at expressing and mechanically checking reachability and required-sequence constraints on modeled topologies; however, their guarantees are strictly limited to the rule battery they encode. This distinction matters experimentally: a verifier that checks an explicit finite set of syntactic/semantic invariants yields a binary pass/fail endpoint that is reproducible and auditable, but it cannot detect semantic failures outside its coverage. Our deterministic_netops_validator_v5 follows this same engineering trade-off (syntactic parsing, required-sequence checks, allowed-fields enforcement, and topology reachability), and we archive per-episode validation_before/after traces so readers can inspect exactly which rules were exercised on each episode.

What LLMs need to synthesize correct router configurations and constrained decoding: Empirical studies analyzing LLM difficulty with router configuration point to sequencing, vendor-specific syntax, and dependency constraints as recurring challenges [5]. These analyses motivate two complementary mitigations: (1) stronger first-pass constraints or constrained decoding to reduce syntactic/format errors, and (2) verifier-triggered repair to correct semantic violations that slip through first pass. Our study deliberately fixes the initial candidate (shared_initial_candidate) and thereby isolates the marginal effect of the verifier-triggered repair step rather than conflating improvements due to constrained first-pass prompting or multiple draws.

Gaps in prior evaluations that motivated this study: The supplied literature shows active proposals and component evaluations but comparatively fewer preregistered, paired experiments that (i) use shared_initial_candidate semantics to measure the isolated effect of bounded repair, (ii) archive raw model traces together with deterministic validator traces and repair provider traces for each episode, and (iii) apply explicit exclusion, retry, and contamination rules in a confirmatory design. Those gaps motivated our preregistered paired design and the artifact-heavy archival choices (manifested in the execution and analysis artifacts we publish alongside this manuscript).

Methodologically relevant syntheses for experimental design: From the surveyed and prototyped prior work we derived three concrete design choices: (1) use a deterministic validator battery (Batfish-style) to obtain an auditable binary endpoint; (2) bound the repair loop (<=1 automated repair call per failed candidate) to maintain a clear paired estimand under shared_initial_candidate semantics; and (3) preregister explicit missingness, retry, and contamination treatments to avoid post hoc analytic choices. Each of these choices has precedent or justification in the cited literature: verifiers as deterministic oracles [4], GVR architectures for regulated settings [3], and benchmark/task construction recommendations that emphasize reproducibility and contamination controls [1,2].

In sum, our related-work synthesis locates this study at the intersection of applied intent→CLI benchmarks, survey-driven mitigation recommendations, GVR methodological proposals, and deterministic verification tooling. The specific empirical contribution is a preregistered, archived paired GVR evaluation under shared_initial_candidate semantics that fills a documented gap in artifact-grounded confirmatory assessments of verifier-triggered repair effectiveness.

## III. Methodology

Preregistration and estimand: The study preregistration is cnsms2026-001-gvr-batfish-prereg-v1 (manifest SHA-256: 46a223492f760950b3d06f475129ce21e52e296c4c2a1e09df3ccd50bec9f891). Primary estimand: paired_success_rate_difference_guarded_minus_baseline under shared_initial_candidate semantics. Under those semantics the identical single sampled initial model candidate is used to compute both outcomes: baseline single-shot candidate → deterministic validator pass/fail, and guarded outcome → same candidate; if it fails, the guarded arm may invoke at most one automated LLM repair call and the final validator pass/fail is recorded. This isolates the effect of the allowed validator-triggered repair step for a fixed initial candidate.

Task bank and sampling: The preregistered task bank deterministically produced 300 intent→CLI pairs (150 Cisco-like; 150 Juniper-like) using a seeded synthetic task generator combined with public NetConfEval-style examples; per-task generator ids, seeds, and manifest entries are archived. Inclusion/exclusion, retry, and missingness rules followed the preregistration: provider/API transient failures allowed up to two automatic retries; pairs where both arms failed due to provider/infrastructure errors were excluded from the primary analysis. The execution manifest records planned_episode_count = 600 and completed_episode_count = 532; missingness and reconciliation artifacts are archived.

Model, orchestration, and sampling disclosure: The declared model was gpt-5-mini (execution/model_configuration.json field declared_model_version = "gpt-5-mini"). Archived configuration fields include max_output_tokens = 1024, maximum_attempts_per_call = 1, provider = openai_responses, and temperature = null (unset). Provider accounting (deterministic_reconciliation.json) records hosted_model_call_event_count = 306, completed_hosted_model_call_event_count = 272, and failed_hosted_model_call_event_count = 34. Because

temperature was unset and no centralized deterministic sampling seed was archived for provider calls, nondeterministic sampling cannot be excluded for recorded model calls; we note this explicitly in the Disclosure and Limitations.

Deterministic validator semantics and scoring rules: The binary oracle used across episodes was deterministic_netops_validator_v5. For each candidate the validator performed: (1) CLI syntactic parsing/normalization; (2) per-task required-sequence checks (expected_sequence vs parsed commands); (3) constrained-field touch checks (allowed_fields, allowed_touched_fields); and (4) reachability/constraint validation on the synthetic topology model. Each episode archives validation_before and validation_after traces containing parsed_command_count, required_command_count, observed_touched_field_count, violation codes, and normalized_configuration; the validator's pass/fail output is the primary binary endpoint. The confirmatory scoring rule was 'full_intent_constraint_validation' as recorded in analysis artifacts.

Repair loop mechanics (bounded, archived): Per the preregistration the repair stage is bounded to at most one automated LLM repair call per task when the initial candidate fails the validator. The repair prompt includes validator failure codes and task constraints and requests a corrected configuration; repair provider traces and repair_prompt_sha256 are archived when invoked. Stage accounting shows shared_initial_generation = 300 and repair = 6 (six repair calls executed). Repair artifacts and validator traces for repaired episodes are included in execution/scoring and representative artifacts described below.

Analysis plan and inferential methods: The prespecified primary test is the exact McNemar two-sided test on paired binary outcomes at $\alpha$=0.05. Interval estimation was preregistered as a paired bootstrap percentile CI (B=10,000, seed=1729) for the paired difference to quantify magnitude and uncertainty. Missingness handling followed the preregistered 'complete_pair_primary' rule: both-arm provider failures were excluded from the primary test; archived sensitivity analyses treat excluded episodes conservatively as failures. Secondary estimands included median_repair_iterations_per_task and median end-to-end latency; per-episode latency was not archived and thus could not be evaluated. We included the paired bootstrap CI alongside McNemar because exact McNemar p-values can be discrete and conservative with few discordant pairs, whereas a bootstrap CI summarizes empirical paired-difference variability (see Results for the observed small-discordant behavior).

## IV. Results

Execution accounting and sample flow: The preregistered plan targeted 300 independent intent→CLI tasks (600 episodes across two conditions). The run recorded planned_episode_count = 600 and completed_episode_count = 532; provider/infrastructure transient failures produced failed_episode_count = 68. Applying the preregistered 'complete_pair_primary' exclusion rules (exclude both-arm provider/infrastructure failures) yielded complete paired units analysed = 266. The analysis pipeline's provider accounting indicates hosted_model_call_event_count = 306 total model-call events across the run, with completed_hosted_model_call_event_count = 272 and failed_hosted_model_call_event_count = 34; stage accounting records shared_initial_generation = 300 and repair = 6.

Primary paired outcomes and contingency counts: For the 266 complete pairs the validator outcomes produced the following contingency counts: n11 = 260 (both baseline and guarded pass), n10 = 5 (guarded pass only), n01 = 0 (baseline pass only), and n00 = 1 (both fail). These counts yield observed per-condition proportions baseline = 260/266 = 0.9774436090 and guarded = 265/266 = 0.9962406015; the paired (guarded - baseline) difference = 0.01879699248. The exact McNemar two-sided test applied to the discordant pair counts (n10 = 5, n01 = 0) produced p = 0.0625. Because the exact two-sided McNemar p for observed discordants is 0.0625 (1/16), it narrowly misses the preregistered $\alpha$ = 0.05 rejection threshold.

Interpreting the small-discordant pattern and the two inferential summaries: The discordant total (n10 + n01 = 5) is small. The exact two-sided McNemar test calculates two-sided probability mass under a Binomial(total=5, p=0.5) for outcomes as or more extreme than the observed (k >= 5 or k <= 0), yielding p = 2 * (0.5^5) = 0.0625. In contrast, the paired bootstrap percentile CI (B = 10,000 resamples; seed = 1729) for the paired difference is [0.0037593985, 0.0375939850] and excludes zero. Both summaries are valid descriptions of uncertainty; the discrete exact test is conservative when discordant counts are few, while the bootstrap CI summarizes empirical resampling variability. We therefore report both: the preregistered hypothesis test (McNemar) did not reject at $\alpha$ = 0.05, while the paired bootstrap CI suggests a small positive effect whose lower bound is barely above zero for the observed sample.

Repair invocation, yield, and derived operational quantities: Repair was invoked in 6 episodes (stage_counts: repair = 6). The contingency pattern (n10 = 5) indicates that at most five of those repair invocations contributed to additional guarded successes among the 266 analysed pairs; the run-level accounting (6 repair calls vs 5 discordant guarded wins) is consistent with a small number of successful repairs and at most one repair that did not produce a guarded pass among the complete pairs. Median repair iterations per task across complete pairs is 0 (most tasks did not require repair). A useful derived metric is the number needed to treat (NNT) interpretation: 1 / absolute_risk_reduction = 1 / 0.01879699248 $\approx$ 53.2, i.e., roughly 53 guarded tasks (under this run's conditions and shared_initial_candidate semantics) were associated with one additional verifier-approved configuration relative to baseline. This NNT is a descriptive operational scalar and should be interpreted in context: with a very high baseline pass rate, the marginal correction yield per repair opportunity is small.

Representative repair case diagnostics: Representative ar-

tifacts illustrate typical failure and repair behavior. Task-000008 (difficulty: "extreme", required_command_count = 9) provides a concrete example: the shared initial candidate contained a truncated final token line ("interface uplink2\n") and failed the validator's parsed_command_count vs required_command_count check; the automated repair call produced a corrected candidate that completed the previously truncated command and included the required edge1 sequence. The guarded final validator trace for task-000008 shows validation_after.valid = true and violation_count = 0. These per-episode validator traces record parsed_command_count, required_command_count, observed_touched_field_count, normalized_configuration, and violation codes, enabling targeted audit of repair logic and verifier coverage for each repaired episode.

Sensitivity to missingness and preregistered conservative handling: Thirty-four both-arm episodes were excluded from the primary confirmatory analysis (complete_pairs = 266) per preregistered rules. A conservative sensitivity analysis that treats those 34 excluded both-arm episodes as failures (complete N = 300) yields baseline = 260/300 = 0.866667 and guarded = 265/300 = 0.883333; the discordant counts among the complete pairs remain n10 = 5, n01 = 0, producing the same exact McNemar p = 0.0625 for the paired test on the complete-pair subset. This shows the primary inferential conclusion (nonrejection under exact McNemar) is robust to preregistered conservative missingness treatment, while the absolute per-condition proportions change due to inclusion of excluded episodes as failures.

Quantitative artifact summaries and reproducibility metadata used in Results: All numerical summaries reported above derive directly from archived analysis artifacts: the paired contingency counts, condition summaries, missingness summary, and provider-call accounting produced the reported totals (complete_pairs = 266; baseline_success_count = 260; guarded_success_count = 265; hosted_model_call_event_count = 306; repair calls = 6; both_missing_pairs = 34). The paired bootstrap used B = 10,000 resamples with seed = 1729 (bootstrap parameters archived) to produce the reported percentile CI. Per-episode validator traces and scoring JSONs include normalized_configuration, parsed_command_count, required_command_count, and violation codes for each episode and can be used to reproduce failure-mode breakdowns or to recalibrate validator rule coverage.

Limitations specific to the observed Results: (1) High baseline pass rate produced few discordant pairs: rare failures reduce the statistical power to detect small absolute improvements under the preregistered exact McNemar test. (2) Repair calls were infrequent (6/300) in this corpus; larger absolute improvements from GVR are more plausible on corpora with lower first-pass accuracy. (3) The NNT and repair yield reported are conditional on shared_initial_candidate semantics (the same initial sample across arms) and the particular model sampling/settings used; different sampling regimes or constrained first-pass prompts could change the yield. (4) Per-episode latency was not archived and thus the preregistered latency estimand could not be evaluated; timing instrumentation is recommended for future runs. (5) Validator coverage is finite: violations outside deterministic_netops_validator_v5's rule battery are unmeasured here.

## V. Discussion

Statistical interpretation and the role of small discordant counts: The paired analysis observed only five discordant pairs favoring the guarded pipeline (n10=5) and zero discordant pairs in the opposite direction (n01=0). Exact McNemar two-sided testing evaluates the symmetry of discordant counts and, for small integer discordant counts, yields discrete p-values; with (b,c)=(5,0) the exact two-sided p = 0.0625 as archived. The preregistered paired bootstrap percentile CI (B=10,000, seed=1729) for the paired difference produces a 95% interval that narrowly excludes zero [0.00376, 0.03759]. Both results are reported because they illuminate different aspects of the same data: the exact McNemar p emphasizes a strict null-hypothesis test under paired discrete outcomes, while the bootstrap CI summarizes empirical uncertainty about the magnitude of the paired difference under resampling of the observed complete pairs (bootstrap settings and seed are archived in analysis_log.jsonl). The observed apparent tension ($p>0.05$, CI excluding zero) arises from discreteness and small discordant counts rather than from contradictory raw facts; readers should interpret the pair together: a small positive effect was observed, but it did not meet the preregistered exact-test decision threshold.

Effect size, power, and practical detectability in context: The preregistered power calculation targeted detecting a 0.10 absolute improvement at 80% power with N=300 pairs. The observed effect ($\approx$0.0188) is far smaller than that target; with complete_pairs = 266 (reduced from planned 300 due to 34 both-arm provider failures) the realized power to detect such a small effect is negligible. In practical terms this experiment demonstrates that when baseline first-pass success is already high ($\approx$97.7%), bounded automated repair that can be invoked at most once per failed candidate will rarely be exercised (6 repairs invoked across 300 planned tasks) and thus can only produce small absolute improvements unless the baseline error rate is larger or the repair stage is more permissive (e.g., allow multiple repair iterations or stronger repair strategies).

Artifact-grounded diagnostics of pipeline failure modes: The archived per-episode validator traces permit direct inspection of why candidates failed and how repairs corrected them. Representative diagnostics include: - Truncation/malformed output: task-000008's shared initial candidate shows a truncated terminal line ("interface uplink2\n") that caused parsed_command_count mismatches; the automated repair produced the missing explicit admin down line and then passed the validator. Repair artifacts record both the validator violation codes and the repair prompt used, enabling replay of the exact repair stimulus. - Required-sequence violations: several failure traces indicate missing or misordered commands relative to the task's required_sequence; these

are precisely the semantic checks the deterministic validator enforces and which repair prompts referenced when invoked. - Constrained-field touch checks: validator traces include observed_touched_field_count and allowed_touched_fields enforcement; failures of this type reflect field-scope violations rather than pure syntax and are detectable deterministically by the validator. These concrete, archived diagnostics show that the validator's rule battery produced meaningful, actionable failure signals that the repair prompt could address in some cases, but also that many initial candidates already satisfied the rule battery and thus were unaffected by the repair stage.

Operational and design implications grounded in execution artifacts: The execution accounting (deterministic_reconciliation.json and provider call logs) shows: hosted_model_call_event_count = 306, completed_hosted_model_call_event_count = 272, failed_hosted_model_call_event_count = 34, shared_initial_generation = 300, and repair = 6. This instrumentation supports several concrete operational inferences: 1) Low marginal utility of bounded repairs on high-accuracy corpora: with few initial failures, a guarded repair stage invoked at most once yields a small absolute improvement; this suggests teams evaluating GVR in practice should calibrate repair budget (number of allowed repair iterations, repair prompt sophistication) to baseline error rates. 2) Auditability and triage: the archived validator traces and scoring JSONs provide a compact, machine-readable audit trail suitable for operator triage and for post-deployment incident analysis; these artifacts concretely demonstrate how deterministic validation can localize a failure cause (syntax vs sequence vs field scope). 3) Latency and throughput unknowns: per-episode timing was not archived and therefore the preregistered latency estimand could not be evaluated; absent timing data, operational latency tradeoffs between invoking a repair and manual review remain unmeasured in this run.

Threats to validity revisited with artifact evidence: Internal validity: the study respected shared_initial_candidate semantics preregistered intent and used identical initial candidates for both arms; as a result, the measured estimand isolates the effect of the validator-triggered repair only. Construct validity: deterministic_netops_validator_v5 enforces a limited, archived rule battery (syntactic parsing, required_sequence checks, constrained_field checks, reachability on the synthetic topology). Semantic failures outside that battery (for example, subtle policy violations not encoded in the validator) would go undetected and are thus absent from the measured endpoint. External validity: the task corpus combined public NetConfEval-style examples and a seeded synthetic generator limited to two vendor dialect clusters; the manuscript therefore does not claim generality to other vendors, larger topologies, or production heterogeneity. Conclusion validity: provider failures reduced the analyzed N (266 complete pairs) and the small number of discordant pairs limits inference for small effects.

Concrete recommendations for future experiments (artifact-grounded): The archived manifests suggest the following preregistered changes would increase diagnostic value and power in follow-on studies: (a) design arms that manipulate first-pass generation (e.g., constrained prompting, multiple independent draws) rather than relying solely on shared_initial_candidate semantics if the goal is to measure effects of prompting or sampling; (b) allow a bounded sequence of repair iterations (>=1) and instrument repair-step efficacy; (c) record per-episode latency (request→generation→validation→repair→final validation) to enable formal latency/operational tradeoff analysis; (d) expand and publish the validator rule battery and add adversarial synthetic tasks specifically crafted to probe known verifier false negatives—per-episode validator traces in this run show that such audits are feasible; and (e) if the target operational environment has high baseline accuracy, power calculations should target smaller effect sizes or alternative estimands (e.g., time-to-triage reduction, human-in-the-loop workload) that better reflect marginal benefits of repair.

Reproducibility and archival diagnostics: All raw responses, provider call traces, validator traces, scoring JSONs, analysis artifacts (paired contingency table, condition summary, missingness summary, contamination summary), deterministic_reconciliation.json, model_configuration.json, and analysis_log.jsonl (bootstrap parameters: B=10,000, seed=1729) are archived and hashed in the evidence bundle. These artifacts permit exact reanalysis of the paired contingency, replay of repair prompts with the same validator failure codes, and targeted inspection of individual failure modes. The provider accounting and stage counts enable independent auditors to confirm that the repair stage executed only six times and that 34 provider calls failed and were excluded per preregistered rules.

Synthesis: The experiment provides an artifact-rich, preregistered measurement of the limited marginal benefit of a tightly bounded GVR pipeline under shared_initial_candidate semantics when baseline first-pass accuracy is high. The archived traces make clear which failure modes the validator detected, which were amenable to one automated repair call, and where further engineering (expanded validator coverage, richer repair strategies, or different experimental arms) would be required to observe larger absolute benefits.

## VI. Limitations

- Shared_initial_candidate semantics: both arms used the same single sampled initial candidate per preregistration; the estimand measures validator-triggered repair benefit for that fixed initial sample and does not evaluate independent first-pass generations or constrained first-pass prompting strategies.
- Missingness and sample reduction: planned N=300 pairs; after infrastructure/provider exclusions 266 pairs were complete and 34 both-arm episodes were excluded per preregistered rules, reducing effective sample size and precision.

- Small discordant counts: primary inference used exact McNemar with few discordants (n10=5, n01=0); conclusions are sensitive to inferential choice and limited power.
- Validator coverage: deterministic_netops_validator_v5 enforces a finite set of syntactic/semantic/reachability rules; semantic violations outside its coverage are possible and unmeasured.
- Runtime nondeterminism and sampling: archived execution/model_configuration.json records temperature = null and no deterministic seed; nondeterministic sampling cannot be excluded for recorded model calls.
- H2 latency not evaluated: per-episode end-to-end latency was not present in archived per-task artifacts and therefore the preregistered latency bound (median <= 60 s) could not be assessed.
- External validity: task corpus derives from public NetConfEval-style examples and a seeded synthetic generator limited to two vendor-dialect clusters; results do not imply universal benefit across other vendors, larger topologies, or production deployments.

## VII. Conclusion

We executed a preregistered paired experiment that evaluated a GVR pipeline (gpt-5-mini + deterministic_netops_validator_v5 + <=1 automated repair) on intent→CLI tasks under shared_initial_candidate semantics. Across 266 analysed pairs the guarded pipeline improved validator pass rate by 0.01879699248 but did not meet the preregistered exact-McNemar $\alpha$=0.05 decision rule (p=0.0625); a paired bootstrap CI (B=10,000, seed=1729) narrowly excludes zero. Repair calls were rare (6/300) and median repair iterations per task = 0. Per-episode latency was not archived and could not be assessed. All execution and analysis artifacts (raw responses, validator traces, provider call accounting, and reconciliation manifests) are archived in the evidence bundle to support reanalysis and follow-on work.

References [1] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?," in Proc. ACM, 2024, doi: 10.1145/3656296. [2] S. Y. Long, J. Tan, B. Mao, F. Tang, Y. Li, M. Zhao, and N. Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models," IEEE Commun. Surveys & Tutorials, 2025, doi: 10.1109/comst.2025.3526606. [3] R. Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments," SSRN, 2026, doi: 10.2139/ssrn.6754899. [4] M. Brown, A. Fogel, D. Halperin, V. Heorhiadi, R. Mahajan, and T. Millstein, "Lessons from the evolution of the Batfish configuration analysis tool," Proc., 2023, doi: 10.1145/3603269.3604866. [5] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, "What do LLMs need to Synthesize Correct Router Configurations?," Proc., 2023, doi: 10.1145/3626111.3628194.

## Disclosure Statement

Disclosure (concise): Declared model: gpt-5-mini (execution/model_configuration.json archived; declared_model_version = "gpt-5-mini"). Orchestration adapter: hosted_netops_gvr_v1. Deterministic validator: deterministic_netops_validator_v5. Preregistration: cnsms2026-001-gvr-batfish-prereg-v1 (manifest SHA-256: 46a223492f760950b3d06f475129ce21e52e296c4c2a1e09 df3ccd50bec9f891). Initial master prompt immutable reference: provenance/master_prompt.txt (SHA-256: 1872df1e 1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Human role and autonomy boundary: before the autonomy lock a human Research Director selected the topic family and approved pre-lock infrastructure development; after the autonomy lock the registered pipeline autonomously performed literature selection, preregistration, experimental execution, analysis, manuscript drafting, AI peer review simulation, and revision. Nondeterminism disclosure: archived execution/model_configuration.json records temperature = null (unset) and no deterministic sampling seed is archived; therefore nondeterministic sampling of model outputs cannot be excluded for the recorded provider calls. Execution and analysis artifacts, logs, and hash manifests are archived in the evidence bundle referenced in the preregistration.

## References


[1] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296

[2] S.Y. Long, Jingjing Tan, Bomin Mao, Fengxiao Tang, Yangfan Li, Ming Zhao, Nei Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", 2025, 10.1109/comst.2025.3526606

[3] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899

[4] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, Todd Millstein, "Lessons from the evolution of the Batfish configuration analysis tool", 2023, 10.1145/3603269.3604866

[5] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194